

CyberAudit

Notifications System

Celery email task : queued / sent / failed cycle - GET /notifications/me/

Project	CyberAudit Platform
Module	apps/notifications/
Model	Notification (UUID PK, linked to user + AuditRequest)
4 notification types	request_received / status_changed / report_ready / generation_failed
Status cycle	queued (default) -> sent failed
Celery task	send_notification_email(notification_id) : autoretry x3, backoff
Retry config	max_retries=3, default_retry_delay=10s, retry_backoff_max=600s
update_fields	status+sent_at on success / status+error_message on failure
Endpoint	GET /api/notifications/me/ : scoped to request.user
request_reference	SerializerMethodField : obj.request.reference or None
Tests	4 tests across 2 classes
Date	June 2026 / Juin 2026

1. Notification Model

1. Modèle Notification

The Notification model represents a transactional email to send to a user, optionally linked to an AuditRequest. It tracks the full delivery lifecycle: queued at creation, transitioning to sent (success) or failed (after exhausting retries).

Le modèle Notification représente un email transactionnel à envoyé à un utilisateur, optionnellement lié à une AuditRequest. Il trace le cycle complet de livraison : file d'attente à la création, puis sent (succes) ou failed (après épuisement des tentatives).

```
class Notification(models.Model):
    class Type(models.TextChoices):
        REQUEST_RECEIVED = 'request_received', 'Request received'
        STATUS_CHANGED = 'status_changed', 'Status changed'
        REPORT_READY = 'report_ready', 'Report ready'
        GENERATION_FAILED = 'generation_failed', 'Generation failed'

    class Status(models.TextChoices):
        QUEUED = 'queued', 'Queued'
        SENT = 'sent', 'Sent'
        FAILED = 'failed', 'Failed'

    id = models.UUIDField(primary_key=True, default=uuid.uuid4,
editable=False)
    user = models.ForeignKey(AUTH_USER_MODEL, on_delete=CASCADE,
related_name='notifications')
    request = models.ForeignKey('audits.AuditRequest', on_delete=CASCADE,
related_name='notifications', null=True,
blank=True)
    type = models.CharField(max_length=32, choices=Type.choices)
    subject = models.CharField(max_length=255)
    message = models.TextField()
    status = models.CharField(choices=Status.choices,
default=Status.QUEUED, db_index=True)
    sent_at = models.DateTimeField(null=True, blank=True)
    error_message = models.TextField(blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        db_table = 'notifications'
        ordering = ['-created_at']
        indexes = [models.Index(fields=['user', '-created_at'])]
```

Field référence / Référence des champs

Field	Type	Purpose EN / FR
id	UUIDField (PK)	UUID v4 auto-generated. Avoids sequential enumeration. UUID v4 auto-génère. Évite l'énumération séquentielle.
user	FK -> User (CASCADE)	Recipient. Deleting a user cascades to all their notifications. Destinataire. Supprimer un user supprime ses notifications en cascade.
request	FK -> AuditRequest (nullable)	Optional link. NULL = system notification not tied to a request. Lien optionnel. NULL = notification système non liée à une demande.
type	CharField (choices)	Semantic type. 4 values. Drives email templates and UI icons. Type sémantique. 4 valeurs. Pilote les templates email et icones UI.
subject	CharField(255)	Email subject line, pre-filled by creator. Objet de l'email, pre-rempli par le créateur.
message	TextField	Email body (plain text). No HTML, plain text only. Corps de l'email (texte brut). Pas de HTML.
status	CharField (indexed)	queued (default) / sent / failed. db_index=True for dashboard queries. queued (default) / sent / failed. db_index pour les requêtes du tableau de bord.
sent_at	DateTimeField (nullable)	Filled by task on success. NULL = not yet sent or failed. Rempli par la tâche en cas de succès. NULL = pas encore envoyé ou échoué.
error_message	TextField (blank)	Last exception string on failure. Preserved across retries. Dernier message d'exception en cas d'échec. Préserve entre les tentatives.

2. Status Lifecycle

2. Cycle de vie du statut

Every Notification starts at queued. The Celery task transitions it to sent on success or failed on unrecoverable error. Retries are automatic (up to 3 times), status stays queued until the final retry exhausts.

Chaque Notification démarre a queued. La tache Celery la fait passer a sent en cas de succès ou d'échec en cas d'erreur irrécupérable. Les nouvelles tentatives sont automatiques (3 fois), le statut reste queued jusqu'à épuisement.

Status	DB row state	Trigger	Detail EN / FR
queued	sent_at=NULL error_message=""	Object creation	Default. Row created before task fires. Stays queued during retries. Default. Ligne créée avant l'envoi de la tâche. Reste rn file d'attente pendant les tentatives.
sent	sent_at=now() error_message=""	send_mail() OK	Task success: status=SENT, sent_at=timezone.now(). update_fields only. Succès +de la tâche : status=SENT, sent_at=timezone.now(). update_fields uniquement.
failed	sent_at=NULL error_message=exc	3 retries exhausted	Task raises after storing error. Celery retries x3, then marks failed. La tache leve apres stockage de l'erreur. Celery retente x3, puis marque failed.

queued -> sent	send_mail() succeeds on any attempt (1st, 2nd, or 3rd). send_mail() reussit lors de n'importe quelle tentative.
queued -> failed	All 3 retries raise. After last retry Celery stops. status stays failed. Les 3 tentatives levent une exception. Celery s'arrete. status reste failed.
sent -> (terminal)	No further transition. sent is a terminal state. Pas de transition ultérieure. sent est un état terminal.
failed -> (terminal)	No automatic retry beyond max_retries=3. Manual re-queue if needed. Pas de tentative automatique au-delà de max_retries=3. Re-queue manuel si nécessaire.

3. Celery Task “send_notification_email”

3. Tâche Celery “send_notification_email”

send_notification_email is a bound Celery task with exponential backoff retry. It fetches the Notification, sends the email via Django's send_mail(), and updates only the changed columns with save(update_fields=[...]). On any SMTP exception it stores the error, updates status to FAILED, then re-raises so Celery can retry.

send_notification_email est une tâche Celery liée avec retry exponentiel. Elle récupère la Notification, envoie l'email via send_mail() de Django, et met à jour uniquement les colonnes modifiées. Sur toute exception SMTP elle stocke l'erreur, met status=FAILED, puis relance pour que Celery puisse retenter.

Task configuration / Configuration de la tâche

Parameter	Value	Effect EN / FR
bind=True	True	self is the task instance (needed for self.retry()). Access to request metadata. self est l'instance de la tâche. Accès aux métadonnées de la requête.
autoretry_for	(Exception,)	Any exception triggers retry automatically. No manual self.retry() needed. Toute exception déclenche une tentative automatique. Pas de self.retry() manuel.
max_retries	3	Up to 3 retries (4 total attempts). After 3rd retry: raises MaxRetriesExceeded. Jusqu'à 3 tentatives (4 au total). Après la 3ème : lève MaxRetriesExceeded.
default_retry_delay	10 seconds	Base delay before first retry. Délai de base avant la première tentative.
retry_backoff	True	Exponential backoff: 10s -> 20s -> 40s... Backoff exponentiel : 10s -> 20s -> 40s...
retry_backoff_max	600 seconds	Caps backoff at 10 minutes. Prevents excessively long delays. Plafonne le backoff à 10 minutes. Évite les retards excessivement longs.

Task implementation / Implementation de la tâche

```
@shared_task(
    bind=True,
    autoretry_for=(Exception,),
    max_retries=3,
    default_retry_delay=10,
    retry_backoff=True,
    retry_backoff_max=600,
)
def send_notification_email(self, notification_id):
    from .models import Notification

    try:
        notif =
Notification.objects.select_related('user').get(pk=notification_id)
    except Notification.DoesNotExist:
        return f'Notification {notification_id} introuvable' # no retry

    try:
        send_mail(
            subject=notif.subject,
            message=notif.message,
            from_email=None, # uses DEFAULT_FROM_EMAIL from settings
            recipient_list=[notif.user.email],
            fail_silently=False,
        )
    except Exception as exc:
        notif.status = Notification.Status.FAILED
        notif.error_message = str(exc)
        notif.save(update_fields=['status', 'error_message'])
        raise # triggers autoretry

    notif.status = Notification.Status.SENT
    notif.sent_at = timezone.now()
    notif.save(update_fields=['status', 'sent_at'])
    return f'sent to {notif.user.email}'
```

Late import of Notification	'from .models import Notification' inside the function body, avoids circular imports at module load time. 'from .models import Notification' dans le corps, évite les imports circulaires au chargement.
select_related('user')	One SQL query for notif+user. Avoids N+1 on email lookup. Une seule requête SQL notif+user. Évite le N+1 sur l'email.
from_email=None	Uses DEFAULT_FROM_EMAIL from Django settings. No hardcoded address. Utilise DEFAULT_FROM_EMAIL de Django. Pas d'adresse codée en dur.
fail_silently=False	Raises on SMTP error. Triggers retry via autoretry_for=(Exception,). Lève une exception SMTP. Déclenche retry via autoretry_for.

save(update_fields=[...])

Only status+sent_at or status+error_message written. No full-row UPDATE. Uniquement les colonnes modifiées. Pas d'UPDATE complet.

4. Serializer & View

4. Serialiseur et vue

NotificationSerializer adds request_reference as a SerializerMethodField, this reads the linked AuditRequest's reference string (DOSSIER-YYYY-NNNN) or returns None if the notification has no request. All fields are read-only: no creation via API.

NotificationSerializer ajoute request_reference comme SerializerMethodField, il lit le champ référence de l'AuditRequest liée (DOSSIER-YYYY-NNNN) ou retourne None si pas de demande. Tous les champs sont read_only : pas de création via l'API.

NotificationSerializer

```
class NotificationSerializer(serializers.ModelSerializer):
    request_reference = serializers.SerializerMethodField()

    class Meta:
        model = Notification
        fields = [
            'id', 'type', 'subject', 'message', 'status',
            'request_reference', 'sent_at', 'created_at',
        ]
        read_only_fields = fields

    def get_request_reference(self, obj):
        return obj.request.reference if obj.request else None
        # obj.request pre-fetched via select_related('request') in view
```

Field in response	Source / note EN / FR
id	UUID string. Unique per notification. Chaîne UUID. Unique par notification.
type	request_received / status_changed / report_ready / generation_failed. One of the four types. Un parmi les 4 types.
subject	Email subject as stored. Shown as notification title in UI. Objet email tel que stocké. Affiché comme titre dans l'UI.
message	Email body (plain text). Corps email (texte brut).
status	queued / sent / failed. en file d'attente / envoyé / échec.
request_reference	SerializerMethodField. 'DOSSIER-2026-0001' or null if no linked request.

	SerializerMethodField. 'DOSSIER-2026-0001' ou null si pas de demande liee.
sent_at	ISO 8601 datetime or null. Null for failed or queued. Datetime ISO 8601 ou null. Null pour en échec ou en file d'attente .
created_at	ISO 8601 creation datetime. Used for ordering (newest first). Datetime de creation ISO 8601. Utilise pour le tri (plus recent en premier).

NotificationListView

```
class NotificationListView(generics.ListAPIView):
    serializer_class = NotificationSerializer
    permission_classes = [IsAuthenticated]

    def get_queryset(self):
        return Notification.objects.filter(
            user=self.request.user,          # scope to current user only
        ).select_related('request')         # avoids N+1 on request_reference
```

generics.ListAPIView	Read-only list. No POST/PUT/DELETE on this endpoint. Liste lecture seule. Pas de POST/PUT/DELETE sur cet endpoint.
filter(user=request.user)	Per-user isolation. User A never sees User B's notifications. Isolation par utilisateur. L'utilisateur A ne voit jamais les notifications de B.
select_related('request')	Pre-fetches AuditRequest for request_reference field. 1 JOIN vs N+1 queries. Pre-charge AuditRequest pour request_reference. 1 JOIN vs N+1 requetes.
Ordering	Model Meta: ordering=['-created_at']. Newest notification first. Ordering dans Meta : newest first.

5. URL Routing & Endpoint Reference

5. Routage URL et référence de l'endpoint

```
# notifications/urls.py -- prefixed with /api/ in config/urls.py
urlpatterns = [
    path('notifications/me/', NotificationListView.as_view(), name='notification-
list-me'),
]
```

URL	Method	Auth	Status	Description EN / FR
GET /api/notifications/me/	GET	Bearer	200, 401	Paginated list of current user's notifications, newest first. Liste paginée des notifications de l'utilisateur, plus récent en premier.

Response example / Exemple de réponse

```
// GET /api/notifications/me/ -- 200 OK
[
  {
    'id': '3fa85f64-5717-4562-b3fc-2c963f66afa6',
    'type': 'status_changed',
    'subject': 'Your audit request status has changed',
    'message': 'Your request DOSSIER-2026-0042 has moved to: in_progress.',
    'status': 'sent',
    'request_reference': 'DOSSIER-2026-0042',
    'sent_at': '2026-06-17T10:00:00Z',
    'created_at': '2026-06-17T09:58:00Z'
  },
  {
    'id': '1a2b3c4d-...',
    'type': 'request_received',
    'subject': 'We received your audit request',
    'message': 'Your request DOSSIER-2026-0042 has been created.',
    'status': 'sent',
    'request_reference': 'DOSSIER-2026-0042',
    'sent_at': '2026-06-17T09:55:00Z',
    'created_at': '2026-06-17T09:54:00Z'
  }
]
```

6. Integration: When Notifications Are Created

6. Intégration : Quand les notifications sont créées

Notifications are NOT created by the notifications app itself, they are created by audits views whenever an AuditRequest is created or its status changes. The pattern: create Notification row -> enqueue send_notification_email.delay(str(notif.id)).

Les notifications ne sont PAS creees par l'app notifications elle-même, elles sont creees par les vues audits lors de la creation d'une AuditRequest ou de son changement de statut. Modèle : créer la ligne Notification -> mettre en file d'attente send_notification_email.delay(str(notif.id)).

Trigger event	Type used	When / By whom EN / FR
AuditRequest created	request_received	Fired in AuditRequestView.perform_create(). Recipient = request.user. Déclenche dans AuditRequestView.perform_create(). Destinataire = request.user.
Status -> in_progress	status_changed	Fired when admin updates status via PATCH. Recipient = request.user. Déclenche quand l'admin met à jour le statut via PATCH.
Status -> completed	status_changed	Same trigger as above with new status value. Même déclencheur que ci-dessus avec la nouvelle valeur de statut.
PDF report generated	report_ready	Fired by Celery PDF task on success. Notifies client report is ready. Déclenche par la tache Celery PDF en cas de succes.
PDF generation failed	generation_failed	Fired by Celery PDF task on failure. Notifies admin of the error. Déclenche par la tache Celery PDF en cas d'échec.

Typical creation + dispatch pattern

```
# In audits/views.py -- perform_create() or status update handler
from apps.notifications.models import Notification
from apps.notifications.tasks import send_notification_email

# 1. Create the Notification row (status=QUEUED by default)
notif = Notification.objects.create(
    user=audit_request.user,
    request=audit_request,
```

```
type=Notification.Type.REQUEST_RECEIVED,  
subject='We received your audit request',  
message=f'Your request {audit_request.reference} has been created.',  
)  
  
# 2. Dispatch the Celery task (async)  
send_notification_email.delay(str(notif.id))  
# Task will update status to SENT or FAILED asynchronously
```

7. Tests : 4 tests across 2 classes

7. Tests : 4 tests en 2 classes

Tests cover model defaults and `__str__`, the Celery task success path (real email delivery to mail.outbox), and the safe handling of missing notification IDs.

Les tests couvrent les valeurs par défaut du modele et `__str__`, le chemin succès de la tache Celery (livraison réelle dans mail.outbox), et la gestion sécurisée des IDs manquants.

Class	Test	DB	Assertion EN / FR
TestNotificationModel	test_default_status_is_queued	Yes	notif.status=='queued', notif.sent_at is None. notif.status=='queued', sent_at est None.
TestNotificationModel	test_str_contains_type_and_email	Yes	str(notif) contains 'status_changed' and client_user.email. str(notif) contient le type et l'email.
TestSendNotificationEmail	test_marks_notification_as_sent	Yes	After task: status=='sent', sent_at not None, len(mail.outbox)==1, correct recipient. Après la tâche : status=='sent', sent_at non null, 1 email dans outbox, bon destinataire.
TestSendNotificationEmail	test_returns_message_if_notification_missing	No	Fake UUID returns string containing 'introuvable'. No exception raised. Faux UUID retourne une chaine avec 'introuvable'. Pas d'exception levée.

Test setup notes / Notes sur la configuration des tests

mail.outbox	Django test email backend captures all sent emails in mail.outbox list. len==1 confirms exactly one email sent. Le backend email de test. Django capture tous les emails dans mail.outbox. len==1 confirme exactement un email.
-------------	---

mail.outbox[0].to	List of recipients: ['user@example.com']. Confirms correct routing. Liste des destinataires. Confirme le bon routage.
refresh_from_db()	Reloads the ORM instance from DB after task runs. Confirms task wrote to DB correctly. Recharge l'instance ORM depuis la DB. Confirme que la tâche a bien écrit.
'introuvable' in result	Missing notification returns a message string, not an exception. Task handles gracefully. Notification manquante retourne une chaîne de message, pas une exception.
client_user fixture	Pytest fixture providing an authenticated User. Defined in conftest.py. Fixture pytest fournissant un User authentifié. Défini dans conftest.py.

8. Security & Design Summary

8. Récapitulatif sécurité et conception

Feature	Detail EN / FR
UUID PK	No sequential ID enumeration. UUID v4 auto-generated. Pas d'énumération d'ID séquentiel. UUID v4 auto-génère.
Per-user filter	filter(user=request.user) in get_queryset(). User A never sees User B's notifications. filter(user=request.user). L'utilisateur A ne voit jamais les notifications de B.
IsAuthenticated	All endpoints require valid Bearer token. No public notification access. Tous les endpoints requierent un token valide. Pas d'accès public.
Read-only API	ListAPIView only. No POST/PUT/DELETE. Notifications created internally only. ListAPIView uniquement. Pas de POST/PUT/DELETE. Notifications créées en interne.
Celery async	Email sent asynchronously. API response not blocked by SMTP latency. Email envoyé de façon asynchrone. La réponse API n'est pas bloquée par la latence SMTP.
Retry + backoff	max_retries=3, exponential backoff up to 600s. Handles transient SMTP errors. max_retries=3, backoff exponentiel jusqu'à 600s. Gère les erreurs SMTP transitoires.
error_message stored	Exception string preserved on failure. Enables admin diagnosis. Chaine d'exception préservée en cas d'échec. Permet le diagnostic par l'admin.
update_fields	Task writes only changed columns, no full-row UPDATE. La tâche écrit uniquement les colonnes modifiées.
select_related	View: select_related('request') avoids N+1 on request_reference. Vue : select_related('request') évite le N+1 sur request_reference.
Nullable request FK	Notifications not tied to AuditRequest are supported (system alerts). Notifications non liées à une AuditRequest supportées (alertes système).